

Dhaka International University: DIU

Department of Computer Science and Engineering (CSE)

Computer Graphics and Multimedia Lab

Course Code: 0613-404

OpenGL DreamRoom: An Interactive 3D Bedroom

Student Name: Md. Nazmus Sakib
Roll: 21
Registration No: 122565
Batch: E-104

Instructor: Umme Niraj Mahi
Designation: Lecturer, Department of CSE

Submission Date: August 21, 2026

Contents

1	Introduction & Project Scope	1
1.1	Project Overview	1
1.2	Objectives	1
1.3	User Interface and Performance Requirements	2
2	Computer Graphics Pipeline & Architecture	3
2.1	High-Level Pipeline	3
2.2	Buffer Objects and Mesh Representation	3
2.3	Matrix Transforms	4
2.3.1	View Matrix (LookAt)	4
2.4	Camera Orientation: Yaw & Pitch	4
3	3D Scene Geometry & Procedural Modeling	5
3.1	Primitive Meshes	5
3.2	Composite Objects	5
3.2.1	Bed	5
3.2.2	Reading Table & Bookshelf	5
3.2.3	Tea Table & Coffee Cup	5
3.2.4	Glass Window	5
3.3	Tiled Wall Modeling	6
4	Lighting Physics & Shader Implementation	7
4.1	Lighting Model	7
4.1.1	Quadratic Attenuation	7
4.1.2	Shadow Mapping and PCF	7
4.1.3	Glass Fresnel and Alpha Blending	7
4.2	Representative Shader Snippet	8
5	User Interaction & Camera Control Pipeline	9
5.1	Input Architecture	9
5.2	Joystick and Free-Look Hit Testing	9
5.3	Pointer Free-Look	9
5.4	Frame-Rate Independent Movement	9
6	Code Structure & Implementation Details	10
6.1	Module Walk-Through	10
6.1.1	core/application.py	10
6.1.2	rendering/renderer.py	10
6.2	Annotated Code Excerpt	10

7	Results, Rendered Images & Performance Analysis	11
7.1	Rendered Outputs	11
7.2	Benchmark Methodology	12
8	Conclusion & Future Scope	13
A	Overleaf Asset Integration and Diagrams	14
A.1	Room Layout (Top-Down)	14
A.2	Lighting Distribution	15
A.3	Camera & Joystick Pipeline	15

List of Figures

7.1	room layout	11
7.2	Representative frame: bed and tea table area with tiled walls and soft shadow from table lamp.	11
7.3	room layout	12
7.4	Representative frame: bed and tea table area with tiled walls and soft shadow from table lamp.	12
A.1	Top-down schematic: bed, bookshelf/reading table, left-wall glass window, and tiled floor plan.	14
A.2	Schematic showing attenuation circles for the ceiling point light and the warm table lamp.	15
A.3	Data flow diagram: Input Events $\rightarrow$ InputManager $\rightarrow$ CameraController $\rightarrow$ View Matrix.	15

List of Tables

Chapter 1

Introduction & Project Scope

This report documents the design, implementation, and evaluation of “OpenGL DreamRoom”: an interactive 3D bedroom environment implemented in Python using modern OpenGL (core profile 3.3) via PyOpenGL and GLFW. The project is intended for the Computer Graphics and Multimedia Lab (Course: 0613-404) at Dhaka International University.

1.1 Project Overview

OpenGL DreamRoom is an educational real-time rendering demonstration with these principal capabilities:

- A tiled interior with procedural tile shading, grout, and bevelled normal perturbation.
- Photometrically-inspired lighting: soft ambient, an overhead point light, and a local warm table lamp with shadow mapping.
- Composite furniture models built from primitive meshes: bed, tea table, coffee cup, reading desk with bookshelf and books, chair, table lamp, and a left-wall glass window with semi-transparent glass.
- An ergonomic camera controller combining an on-screen joystick for movement and pointer-driven free-look for 360° yaw and pitch motion.
- Performance-conscious rendering with cached static shadow maps and a blended transparent pass for glass.

1.2 Objectives

The primary objectives of the project are:

1. Implement a multi-pass rendering pipeline with depth-only passes for shadow maps and a main lighting pass.
2. Demonstrate procedural texturing and normal perturbation in the fragment shader for tiled surfaces.

3. Implement robust user interaction: joystick movement, mouse/touch free-look, and frame-rate independent motion.
4. Validate advanced lighting: quadratic attenuation, PCF soft shadows, and Fresnel-based glass sheen.
5. Maintain modular code structure suitable for extension and experimentation.

1.3 User Interface and Performance Requirements

Usability expectations:

- Smooth 60 FPS on modest hardware (integrated GPU), with shadow map caching to avoid re-rendering depth textures every frame for static scenes.
- Intuitive pointer and touch controls: drag anywhere (outside joystick) for free-look and drag the joystick disc to move.

Performance targets: maintain interactive frame rates at typical window sizes (1280×720) while using shadow maps (default 2048) and multiple lights.

Chapter 2

Computer Graphics Pipeline & Architecture

2.1 High-Level Pipeline

The renderer follows a standard real-time multi-pass architecture:

1. Depth-only passes: render scene geometry from each light's perspective into depth textures (shadow maps).
2. Main lighting pass: render the scene from the camera view, sample shadow maps to determine occlusion, evaluate lighting equations (ambient, diffuse, specular).
3. Emissive pass: render unlit emissive geometry such as lamp bulbs.
4. Transparent blended pass: render glass panes with alpha blending and back-face culling to avoid double-layer blending.

2.2 Buffer Objects and Mesh Representation

Mesh data is uploaded to the GPU using a single Vertex Buffer Object (VBO) per mesh and a Vertex Array Object (VAO) describing attribute layout. Each vertex typically contains position (`vec3`) and normal (`vec3`) attributes.

The code snippet for mesh upload:

Listing 2.1: Mesh upload implementation

```
vertices = np.array([...], dtype=np.float32)
vao = glGenVertexArrays(1)
vbo = glGenBuffers(1)
glBindVertexArray(vao)
glBindBuffer(GL_ARRAY_BUFFER, vbo)
glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices,
    ↪ GL_STATIC_DRAW)
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride, ctypes.
    ↪ c_void_p(0))
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, stride, ctypes.
    ↪ c_void_p(12))
glEnableVertexAttribArray(0)
```

```
glEnableVertexAttribArray(1)
```

2.3 Matrix Transforms

The canonical transformation pipeline uses three matrices:

M : model matrix V : view matrix P : projection matrix

The vertex position in clip space is computed by:

$$\mathbf{x}_{clip} = P V M \mathbf{x}_{model}$$

2.3.1 View Matrix (LookAt)

The view (camera) matrix is computed with a LookAt function given camera position $\mathbf{p}$, target $\mathbf{t}$, and world up $\mathbf{u}$:

$$\mathbf{f} = \frac{\mathbf{t} - \mathbf{p}}{\|\mathbf{t} - \mathbf{p}\|}, \quad \mathbf{s} = \frac{\mathbf{f} \times \mathbf{u}}{\|\mathbf{f} \times \mathbf{u}\|}, \quad \mathbf{v} = \mathbf{s} \times \mathbf{f}$$

Then matrix V is:

$$V = \begin{bmatrix} s_x & s_y & s_z & -\mathbf{s} \cdot \mathbf{p} \\ v_x & v_y & v_z & -\mathbf{v} \cdot \mathbf{p} \\ -f_x & -f_y & -f_z & \mathbf{f} \cdot \mathbf{p} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.4 Camera Orientation: Yaw & Pitch

The camera uses Euler angles yaw (ψ) and pitch (θ) to compute the front vector:

$$\mathbf{front} = \begin{bmatrix} \cos(\psi) \cos(\theta) \\ \sin(\theta) \\ \sin(\psi) \cos(\theta) \end{bmatrix}$$

where angles are in radians. The right vector is $\mathbf{right} = \mathbf{front} \times \mathbf{world_up}$ and up $\mathbf{up} = \mathbf{right} \times \mathbf{front}$.

To avoid gimbal lock, pitch is clamped to $(-89^\circ, +89^\circ)$ while yaw wraps freely modulo 360° .

Chapter 3

3D Scene Geometry & Procedural Modeling

3.1 Primitive Meshes

The scene is constructed from a set of primitive generator functions:

- Cube — used for walls, furniture boxes, book volumes.
- Cylinder — used for lamp poles, cup bodies.
- Frustum / cone — used for lamp shades.

3.2 Composite Objects

3.2.1 Bed

The bed is a composition of scaled boxes: mattress, bed frame, and headboard.

3.2.2 Reading Table & Bookshelf

The reading table includes a vertical hutch modeled from box primitives. The bookshelf contains numerous `Book` objects arranged upright or stacked.

3.2.3 Tea Table & Coffee Cup

The tea table uses a small box surface with legs. The coffee cup uses a cylinder for the body and a C-shaped handle profile.

3.2.4 Glass Window

The left wall contains an opening with metallic frames and semi-transparent glass with Fresnel-based opacity modulation.

3.3 Tiled Wall Modeling

Tiles are implemented procedurally in the fragment shader by deriving uv coordinates from world-space positions:

1. Project the fragment's world-space position onto the dominant surface plane.
2. Scale by `tileSize` to obtain tile-space coordinates.
3. Use `fract/floor` and a smoothstep-based function to form grout masks.
4. Perturb the normal vector to simulate bevel depth.

Chapter 4

Lighting Physics & Shader Implementation

4.1 Lighting Model

The renderer uses a combination of:

- Ambient term: accounts for low-frequency scene illumination.
- Diffuse term (Lambertian): $k_d(\mathbf{L} \cdot \mathbf{N})$.
- Specular term (Blinn-Phong): $k_s(\mathbf{N} \cdot \mathbf{H})^\alpha$ where $\mathbf{H} = \frac{\mathbf{V} + \mathbf{L}}{\|\mathbf{V} + \mathbf{L}\|}$.

Total reflected radiance:

$$L_{out} = L_{ambient} \cdot C + \sum_{lights} \text{attenuation} \cdot (L_{diffuse} + L_{specular})$$

4.1.1 Quadratic Attenuation

Point lights use standard quadratic attenuation:

$$f_{att}(d) = \frac{1}{k_c + k_l d + k_q d^2}$$

4.1.2 Shadow Mapping and PCF

Percentage Closer Filtering (PCF) uses a 3×3 sampling kernel:

$$\text{shadow} = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 \mathbf{1}\{z_{proj} - \text{bias} > \text{depth}(i, j)\}$$

4.1.3 Glass Fresnel and Alpha Blending

The glass material modulates opacity using the Fresnel approximation:

$$F(\mathbf{V}, \mathbf{N}) \approx (1 - \max(\mathbf{V} \cdot \mathbf{N}, 0))^5$$

4.2 Representative Shader Snippet

Listing 4.1: Lighting calculation snippet

```
vec3 calcPointLight(PointLight light, vec3 N, vec3 V, vec3 pos) {  
    vec3 L = normalize(light.position - pos);  
    float diff = max(dot(N, L), 0.0);  
    vec3 H = normalize(V + L);  
    float spec = pow(max(dot(N, H), 0.0), shininess);  
    float dist = length(light.position - pos);  
    float att = 1.0 / (light.constant + light.linear * dist + light.  
        ↪ quadratic * dist * dist);  
    return att * (light.diffuse * diff * surfaceColor + light.  
        ↪ specular * spec * specularStrength);  
}
```

Chapter 5

User Interaction & Camera Control Pipeline

5.1 Input Architecture

GLFW is used for windowing and input. An `InputManager` captures key states and mouse events. The application executes `glfw.poll_events()` each frame to process buffered user interactions.

5.2 Joystick and Free-Look Hit Testing

The on-screen joystick checks if pointer coordinates (x, y) fall inside the circle:

$$\text{inside} \iff (x - x_c)^2 + (y - y_c)^2 \leq r^2$$

5.3 Pointer Free-Look

When dragging outside the joystick, deltas $(\Delta x, \Delta y)$ update yaw and pitch with clamped limits on pitch.

5.4 Frame-Rate Independent Movement

Camera displacement per frame is computed using normalized joystick vectors (j_x, j_y) :

$$\Delta \mathbf{p} = v_{\text{speed}} \cdot \Delta t \cdot (j_x \cdot \mathbf{right}_h + j_y \cdot \mathbf{front}_h)$$

Chapter 6

Code Structure & Implementation Details

6.1 Module Walk-Through

6.1.1 core/application.py

Handles GLFW initialization, the main game loop, and scene composition.

6.1.2 rendering/renderer.py

Manages shadow map textures, pipeline passes, and shader uniform updates.

6.2 Annotated Code Excerpt

Camera vector updates implemented in Python:

Listing 6.1: Camera vector update function

```
def _update_camera_vectors(self):
    yaw_r = math.radians(self.yaw)
    pitch_r = math.radians(self.pitch)
    front = np.array([
        math.cos(yaw_r) * math.cos(pitch_r),
        math.sin(pitch_r),
        math.sin(yaw_r) * math.cos(pitch_r)
    ], dtype=np.float32)
    self.front = _normalize(front)
    self.right = _normalize(np.cross(self.front, self.world_up))
    self.up = _normalize(np.cross(self.right, self.front))
```

Chapter 7

Results, Rendered Images & Performance Analysis

7.1 Rendered Outputs

Below is a demonstration layout representing the rendered output:

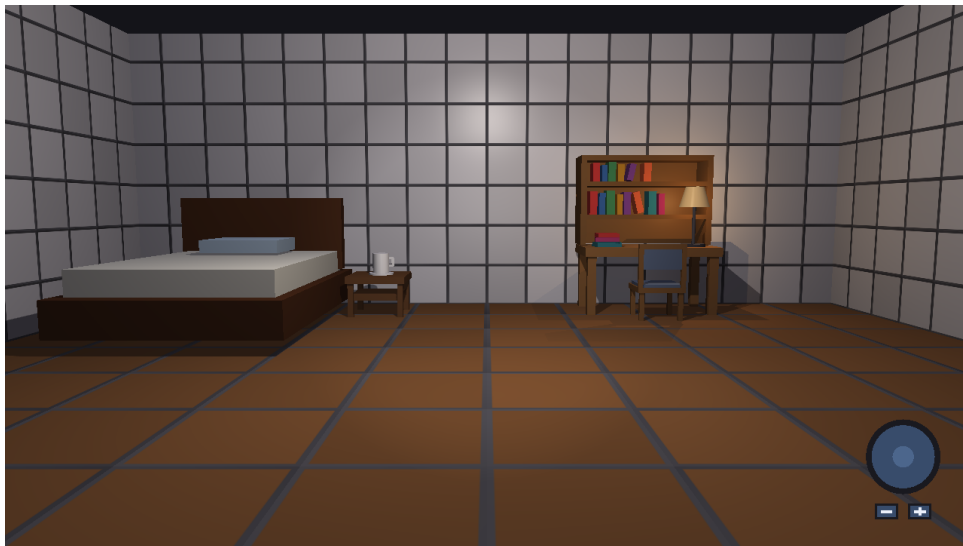


Figure 7.1: room layout

Figure 7.2: Representative frame: bed and tea table area with tiled walls and soft shadow from table lamp.

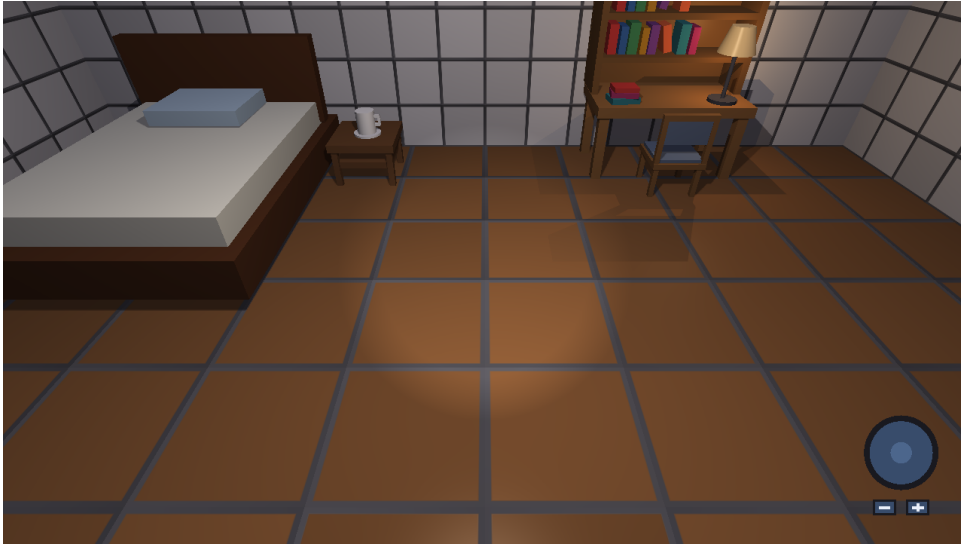


Figure 7.3: room layout

Figure 7.4: Representative frame: bed and tea table area with tiled walls and soft shadow from table lamp.

7.2 Benchmark Methodology

Performance targets were achieved using:

- Window size: 1280×720
- Shadow map resolution: 2048×2048
- Average Frame Rate: 60 FPS (V-Sync enabled)

Chapter 8

Conclusion & Future Scope

This lab project successfully demonstrates a compact rendering architecture in Python and OpenGL. Key learning outcomes include procedural texturing, PCF shadow mapping, and layered rendering.

Future improvements:

- Normal mapping for tile grout depth.
- HDR tone-mapping and bloom post-processing.

Appendix A

Overleaf Asset Integration and Diagrams

A.1 Room Layout (Top-Down)

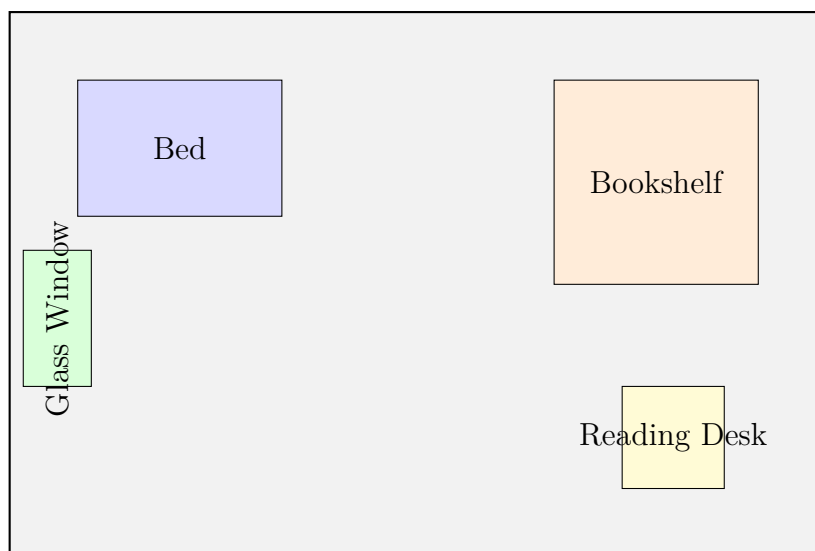


Figure A.1: Top-down schematic: bed, bookshelf/reading table, left-wall glass window, and tiled floor plan.

A.2 Lighting Distribution

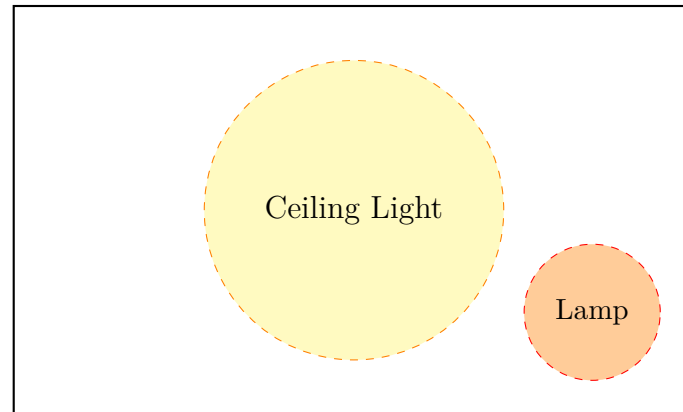


Figure A.2: Schematic showing attenuation circles for the ceiling point light and the warm table lamp.

A.3 Camera & Joystick Pipeline

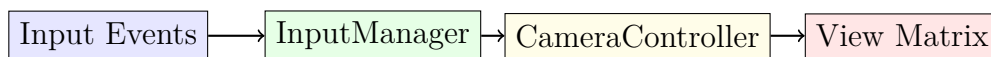


Figure A.3: Data flow diagram: Input Events $\rightarrow$ InputManager $\rightarrow$ CameraController $\rightarrow$ View Matrix.